

THE LAST DANCE

Hack The Box • Cryptography • Easy

Prepared By	Emmanuel Mwangi
Date	September 2026
Machine Author(s)	ckrielle
Category / Difficulty	Cryptography / Easy

Portfolio / CTF Lab Documentation

1. Synopsis

The Last Dance is a cryptography challenge that demonstrates the security risks associated with reusing the same nonce with the same key when using a stream cipher. The challenge provides a Python source file (source.py) and an output file (out.txt). The output contains a nonce, the ciphertext of a known plaintext message, and the ciphertext of the hidden flag.

The challenge uses ChaCha20, a symmetric stream cipher. By analysing the source code, it can be determined that both the known message and the flag are encrypted using the same key and nonce. Because ChaCha20 generates the same keystream when the same key and nonce are reused, the known plaintext and its ciphertext can be used to recover the keystream. The recovered keystream can then be used to decrypt the flag.

Skills Required

- Basic Python programming
- Cryptography fundamentals
- Understanding of XOR operations

Skills Learned

- Symmetric cryptography
- Stream cipher concepts
- XOR-based encryption and decryption
- Identifying nonce-reuse vulnerabilities
- Basic cryptographic analysis using CyberChef

2. Initial Analysis

After downloading and extracting the challenge files, two files are provided:

```
source.py
out.txt
```

The source.py file contains the encryption logic, while out.txt contains the generated nonce and ciphertext values.

2.1 Analysing source.py

The script imports the ChaCha20 cipher, the hidden flag, and Python's os module:

```
from Crypto.Cipher import ChaCha20
from secret import FLAG
import os
```

The challenge defines an `encryptMessage()` function that takes a message, a key, and a nonce:

```
def encryptMessage(message, key, nonce):
    cipher = ChaCha20.new(key=key, nonce=iv)
    ciphertext = cipher.encrypt(message)
    return ciphertext
```

One coding issue can be observed here: the function accepts a parameter named `nonce`, but the code uses the variable `iv` instead. In this challenge, `iv` is the value being used as the ChaCha20 nonce.

The script then creates a known plaintext message:

```
message = b"Our counter agencies have intercepted your messages and a lot "
message += b"of your agent's identities have been exposed. In a matter of "
message += b"days all of them will be captured"
```

Next, a random 32-byte key and 12-byte nonce are generated:

```
key, iv = os.urandom(32), os.urandom(12)
```

The important observation is that the same key and iv are then used to encrypt both the known message and the flag:

```
encrypted_message = encryptMessage(message, key, iv)
encrypted_flag = encryptMessage(FLAG, key, iv)
```

Critical Observation

The same key and nonce are reused for two different messages. With a stream cipher such as ChaCha20, reusing the same keystream creates a cryptographic weakness.

3. Understanding the Vulnerability

ChaCha20 is a stream cipher. It encrypts data by combining the plaintext with a generated keystream using XOR.

```
Plaintext XOR Keystream = Ciphertext
```

For the known message:

```
P1 XOR K = C1
```

Where `P1` is the known plaintext, `K` is the ChaCha20 keystream, and `C1` is the encrypted known message.

For the flag:

```
P2 XOR K = C2
```

Where `P2` is the unknown flag, `K` is the same keystream, and `C2` is the encrypted flag.

Because the same key and nonce are reused, the same keystream is generated for both encryptions. The keystream can therefore be recovered from the known plaintext and ciphertext:

```
P1 XOR C1 = K
```

The actual ChaCha20 key is not recovered. Instead, the attack recovers the keystream generated from the key and nonce. Once the keystream is known, it can be used against the encrypted flag:

```
C2 XOR K = P2
```

Substituting the recovered keystream gives:

```
P2 = C2 XOR P1 XOR C1
```

Key Takeaway

FLAG = Known Plaintext XOR Ciphertext1 XOR Ciphertext2

4. Analysing out.txt

Running the Python script produces out.txt. The output contains three values:

```
Nonce  
Encrypted known message  
Encrypted flag
```

The values are written in hexadecimal format using:

```
data = iv.hex() + "\n" + encrypted_message.hex() + "\n" + encrypted_flag.hex()
```

The nonce is therefore available to the attacker, while the encryption key remains secret. The known plaintext is already present in the Python source code, and its corresponding ciphertext is provided in out.txt. This gives us the information required to recover the reused keystream.

5. Solution Approach

Step 1 — Identify the known plaintext

The plaintext is provided directly in source.py:

```
Our counter agencies have intercepted your messages and a lot  
of your agent's identities have been exposed. In a matter of  
days all of them will be captured
```

Step 2 — Obtain the corresponding ciphertext

The encrypted version of the known message is provided as the second line of out.txt. This is referred to as Ciphertext1.

Step 3 — Recover the keystream

Because Plaintext XOR Keystream = Ciphertext, XORing the known plaintext with Ciphertext1 gives the reused keystream.

```
Known Plaintext XOR Ciphertext1 = Keystream
```

Step 4 — Obtain the encrypted flag

The third line of out.txt contains the encrypted flag, referred to as Ciphertext2.

Step 5 — Recover the flag

XORing Ciphertext2 with the recovered keystream reveals the plaintext flag.

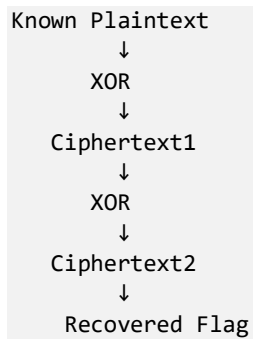
```
Ciphertext2 XOR Keystream = FLAG
```

Equivalent form:

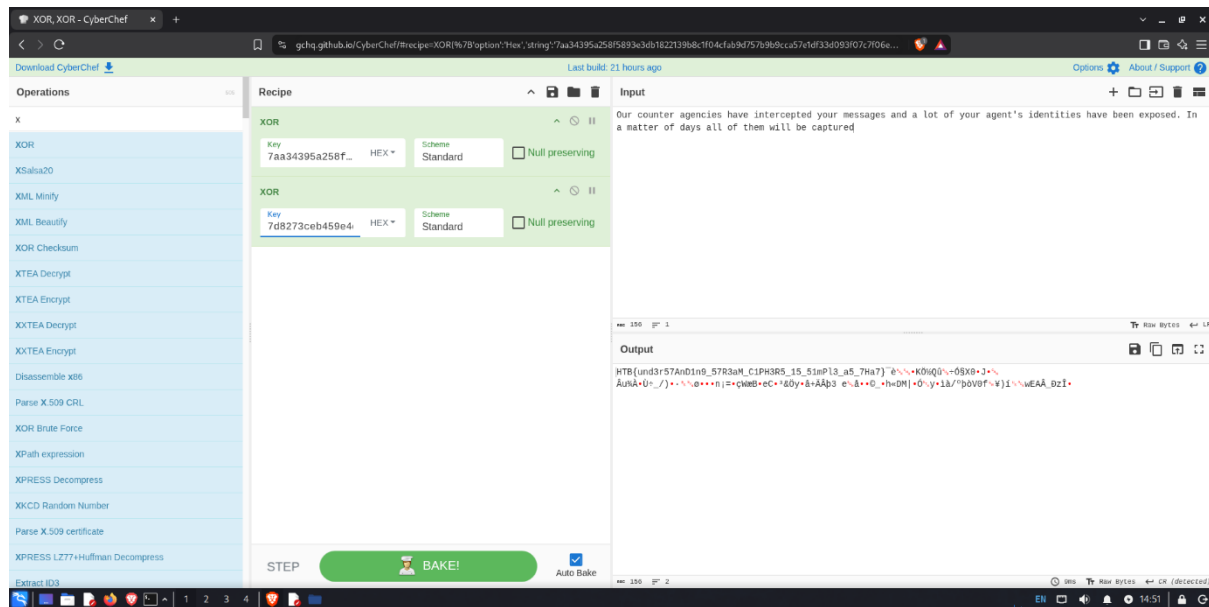
```
FLAG = Known Plaintext XOR Ciphertext1 XOR Ciphertext2
```

6. Using CyberChef

CyberChef can be used to perform the XOR operations required to recover the flag. The process is:



The known plaintext is entered into CyberChef and XORed with the first ciphertext. The resulting keystream is then XORed with the encrypted flag. The final output reveals the plaintext flag.



7. Result

The flag was successfully recovered by exploiting the reuse of the same ChaCha20 key and nonce:

HTB{und3r57And1n9_57R3aM_C1PH3R5_15_51mPl3_a5_7Ha7}

8. Lessons Learned

This challenge demonstrated the importance of correct nonce management when using stream ciphers. Reusing the same nonce with the same key across multiple messages can cause keystream reuse, which may allow an attacker to recover plaintext when a single plaintext/ciphertext pair is known.

The challenge also reinforced the importance of understanding XOR operations and analysing source code before attempting an attack.

9. Security Recommendation

When using ChaCha20, a unique nonce should be used for each encryption operation with a given key.

Unsafe pattern:

```
Same Key + Same Nonce + Message 1  
Same Key + Same Nonce + Message 2
```

Safer pattern:

```
Same Key + Nonce 1 + Message 1  
Same Key + Nonce 2 + Message 2
```

Nonce uniqueness prevents the same keystream from being reused across different messages.

10. Conclusion

The Last Dance demonstrates how a cryptographic implementation can become vulnerable even when a strong encryption algorithm such as ChaCha20 is being used. The weakness was not a break of ChaCha20 itself, but the incorrect reuse of the same key and nonce. By recognising this flaw and applying XOR operations to the known plaintext and ciphertexts, the encrypted flag could be recovered.

This challenge provided practical experience in analysing Python cryptographic code, identifying a nonce-reuse vulnerability, understanding stream-cipher keystreams, and using CyberChef to perform the required XOR operations.